Rapport de Projet

Optimisation du Problème du Voyageur de Commerce (TSP)

Étude comparative d'une Heuristique Gloutonne et du Recuit Simulé

Réalisé par :
TAKI Hamza

15 mars 2026

Table des matières

1 Introduction **2**
1.1 Illustration du problème 2
1.2 Applications réelles 2
1.3 Complexité du problème 3
1.4 Motivation et approche 3
1.5 Objectif du projet 4

2 Description du Problème du Voyageur de Commerce **4**

3 Heuristique gloutonne : Plus Proche Voisin **5**
3.1 Principe de l'algorithme 5
3.2 Avantages et limites 5

4 Métaheuristique : Recuit Simulé **5**
4.1 Principe de l'algorithme 6

5 Voisinage utilisé : mouvement 2-opt **6**

6 Données expérimentales **7**

7 Résultats expérimentaux **7**

8 Influence des paramètres du recuit simulé **8**
8.1 Influence du nombre d'itérations 9
8.2 Influence du facteur de refroidissement 10

9 Analyse des résultats **10**

10 Conclusion **11**

1. Introduction

Le Problème du Voyageur de Commerce (TSP pour *Travelling Salesman Problem*) est l'un des problèmes les plus célèbres et les plus étudiés en optimisation combinatoire et en recherche opérationnelle.

Le principe du problème est relativement simple : un voyageur doit visiter un ensemble de villes exactement une fois chacune, puis revenir à son point de départ, tout en minimisant la distance totale parcourue.

Malgré sa simplicité apparente, ce problème devient extrêmement complexe lorsque le nombre de villes augmente. Le TSP constitue ainsi un problème de référence pour tester et comparer les performances de nombreux algorithmes d'optimisation.

1.1 Illustration du problème

La Figure suivante illustre un exemple simple du problème du voyageur de commerce. Chaque point représente une ville et les arêtes représentent les trajets possibles entre les villes.

L'objectif est de trouver la tournée la plus courte qui visite toutes les villes une seule fois.

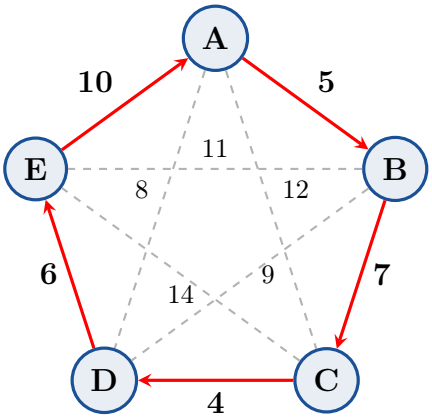


FIGURE 1 – Illustration d'un graphe complet pour le TSP avec une tournée hamiltonienne mise en évidence en rouge (Coût total = 32).

1.2 Applications réelles

Le TSP possède de nombreuses applications dans différents domaines industriels et scientifiques. Parmi les applications les plus importantes, on peut citer :

- la planification des tournées de livraison dans la logistique
- l'optimisation des trajets de transport
- la fabrication de circuits électroniques
- l'optimisation de réseaux de télécommunications
- la robotique et la planification de trajectoires

Dans tous ces cas, l'objectif est de minimiser une distance, un coût ou un temps de déplacement.

1.3 Complexité du problème

La principale difficulté du TSP provient du nombre extrêmement élevé de solutions possibles.

Pour un problème comportant n villes, le nombre de tournées différentes possibles est donné par la formule suivante :

$$\frac{(n-1)!}{2}$$

Cette valeur augmente très rapidement lorsque le nombre de villes augmente, ce qui rend impossible l'exploration exhaustive de toutes les solutions lorsque n devient grand.

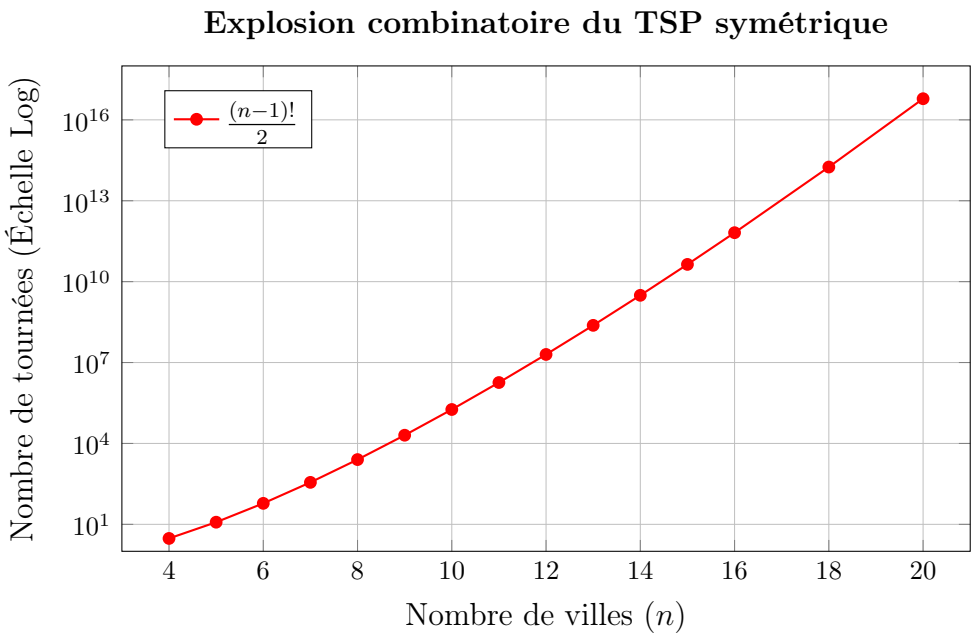


FIGURE 2 – Croissance du nombre de solutions possibles pour le TSP symétrique en fonction du nombre de villes.

Par exemple :

Nombre de villes	Nombre de tournées possibles
10	181 440
15	43 milliards
20	6.08×10^{16}
50	$\approx 10^{62}$

Cette croissance exponentielle explique pourquoi le TSP appartient à la classe des problèmes **NP-difficiles**.

1.4 Motivation et approche

Étant donné la difficulté de résoudre exactement le TSP pour des instances de grande taille, les chercheurs utilisent souvent des méthodes approchées appelées :

- heuristiques

- métaheuristiques

Ces méthodes ne garantissent pas toujours la solution optimale, mais elles permettent généralement d'obtenir des solutions très proches de l'optimum dans un temps de calcul raisonnable.

1.5 Objectif du projet

L'objectif de ce projet est d'étudier et de comparer deux approches différentes pour résoudre le problème du voyageur de commerce :

1. une **heuristique gloutonne** appelée *Plus Proche Voisin* (*Nearest Neighbor*), qui construit une solution rapidement.
2. une **métaheuristique** appelée *Recuit Simulé* (*Simulated Annealing*), qui améliore progressivement la solution initiale.

Les expériences seront réalisées sur plusieurs instances issues de la base de données **TSPLIB**. Une analyse expérimentale sera menée afin d'étudier :

- la qualité des solutions obtenues
- le gain apporté par la métaheuristique
- l'influence de certains paramètres du recuit simulé (température initiale, facteur de refroidissement, nombre d'itérations par palier)

Les résultats seront présentés sous forme de tableaux et de graphiques afin d'évaluer les performances des différentes méthodes étudiées.

2. Description du Problème du Voyageur de Commerce

Le problème du voyageur de commerce peut être modélisé de manière formelle à l'aide d'un graphe complet pondéré.

Soit un graphe $G = (V, E)$ où :

- V représente l'ensemble des villes à visiter
- E représente les arêtes reliant les villes
- chaque arête possède un poids correspondant à la distance entre deux villes

L'objectif est de déterminer un cycle hamiltonien de coût minimal.

Mathématiquement, la fonction objectif peut être exprimée par :

$$\min \sum_{i=1}^{n} d(v_i, v_{i+1})$$

où :

— v_i représente la ville visitée à l'étape i

— $d(v_i, v_j)$ représente la distance entre deux villes

Le nombre de solutions possibles pour n villes est :

$$(n-1)!$$

Cette croissance factorielle explique pourquoi les méthodes exactes deviennent rapidement impraticables pour les grandes instances.

3. Heuristique gloutonne : Plus Proche Voisin

L'heuristique du plus proche voisin (*Nearest Neighbor*) est une méthode constructive simple et rapide pour générer une solution initiale du problème du voyageur de commerce.

Le principe de cette méthode est le suivant : à partir d'une ville initiale, l'algorithme sélectionne à chaque étape la ville non encore visitée la plus proche.

3.1 Principe de l'algorithme

L'algorithme fonctionne selon les étapes suivantes :

1. choisir une ville de départ
2. ajouter la ville la plus proche non encore visitée
3. répéter jusqu'à ce que toutes les villes soient visitées
4. retourner à la ville de départ

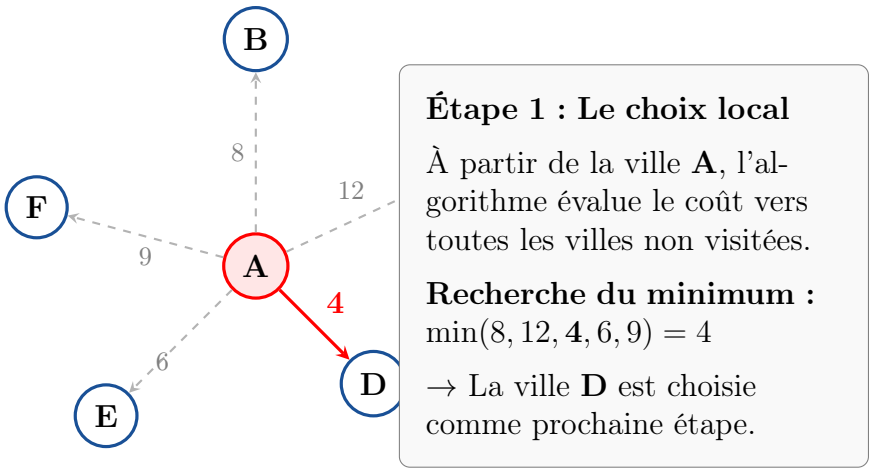


FIGURE 3 – Principe de l'heuristique du plus proche voisin : à chaque itération, l'algorithme sélectionne systématiquement le nœud adjacent de poids minimum.

3.2 Avantages et limites

Cette méthode est très rapide et facile à implémenter. Cependant, elle ne garantit pas l'obtention d'une solution optimale, car les décisions prises localement ne sont pas toujours optimales globalement.

Dans ce projet, cette heuristique est utilisée pour générer une solution initiale qui sera ensuite améliorée par une métaheuristique.

4. Métaheuristique : Recuit Simulé

Le recuit simulé (*Simulated Annealing*) est une métaheuristique inspirée d'un processus physique utilisé en métallurgie appelé recuit.

Le principe consiste à chauffer un matériau puis à le refroidir progressivement afin d'obtenir une structure cristalline stable correspondant à un état d'énergie minimale.

4.1 Principe de l'algorithme

Dans le contexte de l'optimisation, la solution correspond à l'état du système et la fonction objectif correspond à l'énergie.

L'algorithme explore différentes solutions voisines et accepte :

— toujours une solution meilleure

— parfois une solution moins bonne avec une certaine probabilité

Cette probabilité est donnée par la formule de Metropolis :

$$P = e^{-\Delta/T}$$

où :

— Δ représente la variation du coût

— T représente la température

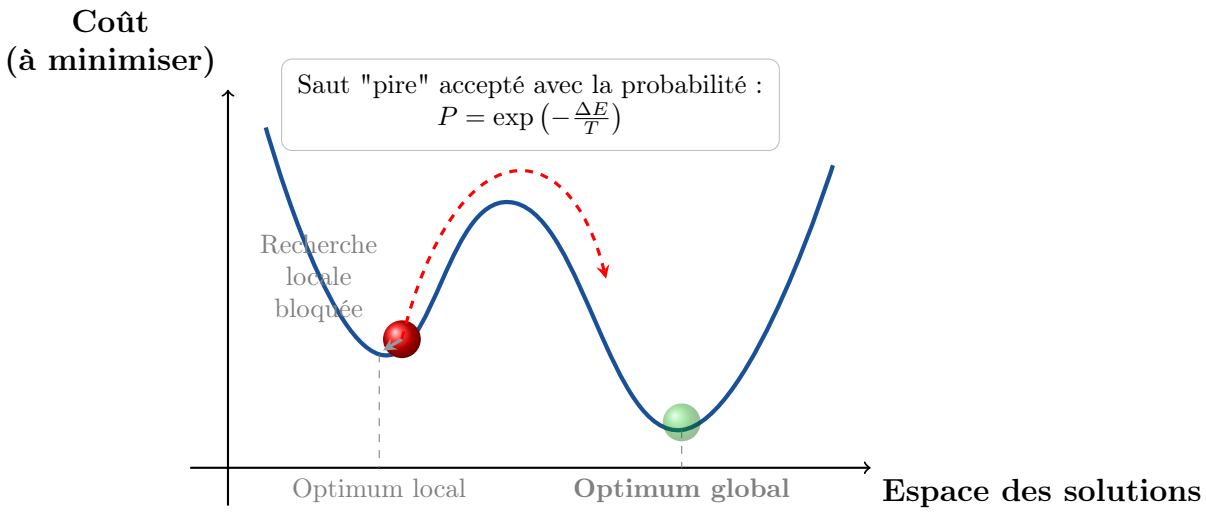


FIGURE 4 – Illustration du paysage d'énergie du Recuit Simulé. Contrairement à une heuristique classique qui reste piégée dans un optimum local, le recuit simulé accepte des dégradations temporaires pour franchir les barrières et atteindre l'optimum global.

La température diminue progressivement au cours de l'algorithme, ce qui réduit la probabilité d'accepter des solutions moins bonnes.

5. Voisinage utilisé : mouvement 2-opt

Afin de générer de nouvelles solutions dans le recuit simulé, nous utilisons l'opérateur de voisinage appelé **2-opt**.

Cet opérateur consiste à supprimer deux arêtes de la tournée et à reconnecter les segments de manière différente afin d'obtenir une nouvelle tournée.

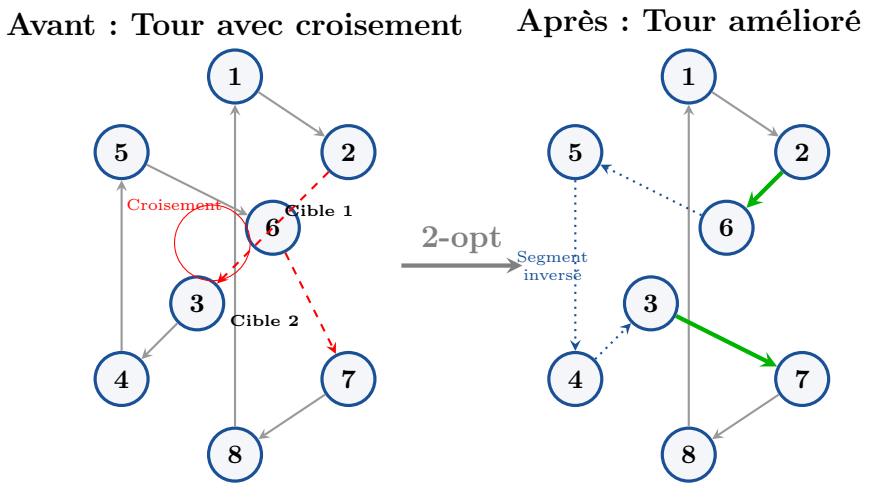


FIGURE 5 – Illustration du principe du mouvement 2-opt. Deux arêtes cibles (en rouge) sont supprimées pour un croisement détecté, et remplacées par deux nouvelles arêtes (en vert), ce qui nécessite l'inversion du sens de parcours du segment de chemin intermédiaire (en bleu pointillés).

Le mouvement 2-opt permet souvent de réduire la distance totale de la tournée en supprimant les croisements entre les arêtes.

6. Données expérimentales

Les expérimentations ont été réalisées sur plusieurs instances issues de la bibliothèque **TSPLIB**, qui constitue une référence standard pour l'évaluation des algorithmes de résolution du TSP.

Les instances utilisées dans ce projet incluent notamment :

- berlin52
- eil51
- eil76
- kroA100
- bier127
- ch150
- a280

Chaque instance contient un ensemble de coordonnées représentant les villes ainsi que les distances entre celles-ci.

Ces instances permettent d'évaluer la performance des algorithmes sur des problèmes de différentes tailles.

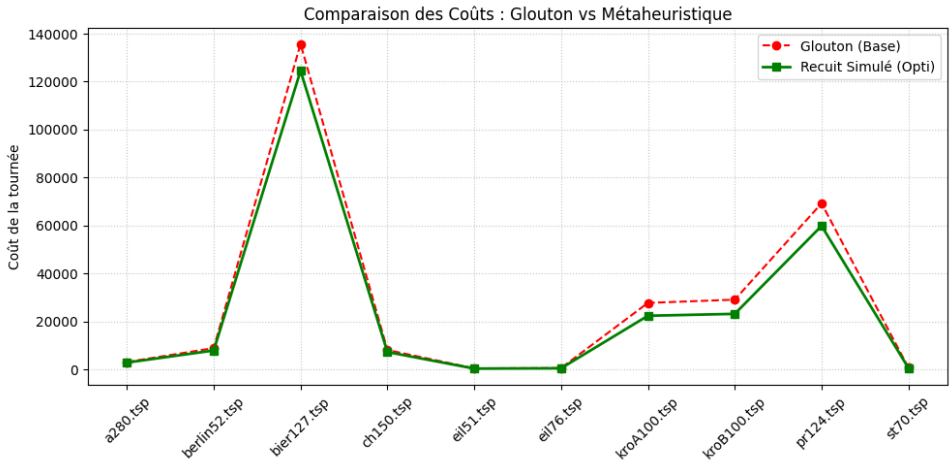
7. Résultats expérimentaux

Les résultats obtenus permettent de comparer les performances de l'heuristique gloutonne et du recuit simulé.

Les critères étudiés sont :

— le coût de la tournée
— le temps d'exécution
— le gain obtenu grâce à la métaheuristique

Instance	Cout_Glouton	Cout_Meta_Optimal	Gain(%)
a280.tsp	3157	2930	7.19
berlin52.tsp	8980	7977	11.17
bier127.tsp	135737	124531	8.26
ch150.tsp	8191	7328	10.54
eil51.tsp	511	438	14.29
eil76.tsp	642	558	13.08
kroA100.tsp	27807	22447	19.28
kroB100.tsp	29158	23236	20.31
pr124.tsp	69297	59897	13.56
st70.tsp	830	703	15.3



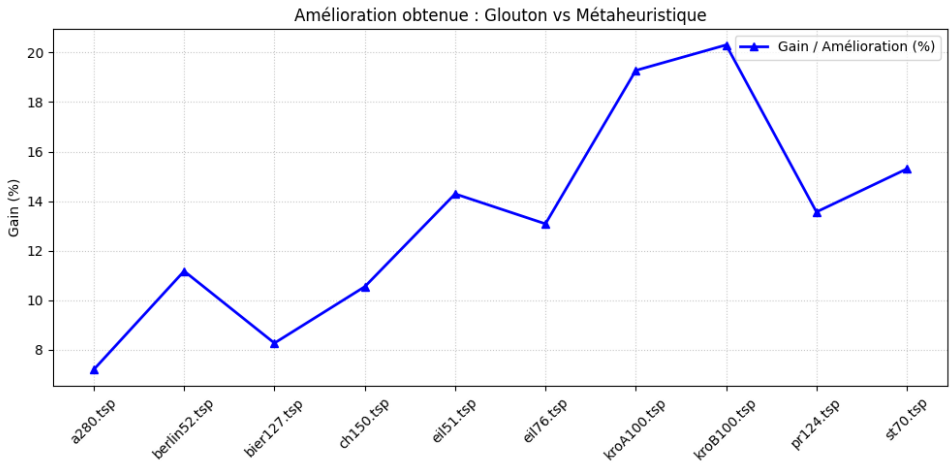


FIGURE 6 – Comparaison des coûts et gain entre l'heuristique gloutonne et le recuit simulé

Les résultats montrent que le recuit simulé permet généralement d'améliorer la solution obtenue par l'heuristique gloutonne.

8. Influence des paramètres du recuit simulé

Les performances du recuit simulé dépendent fortement du choix de certains paramètres.

Dans ce projet, nous avons étudié l'influence des paramètres suivants :

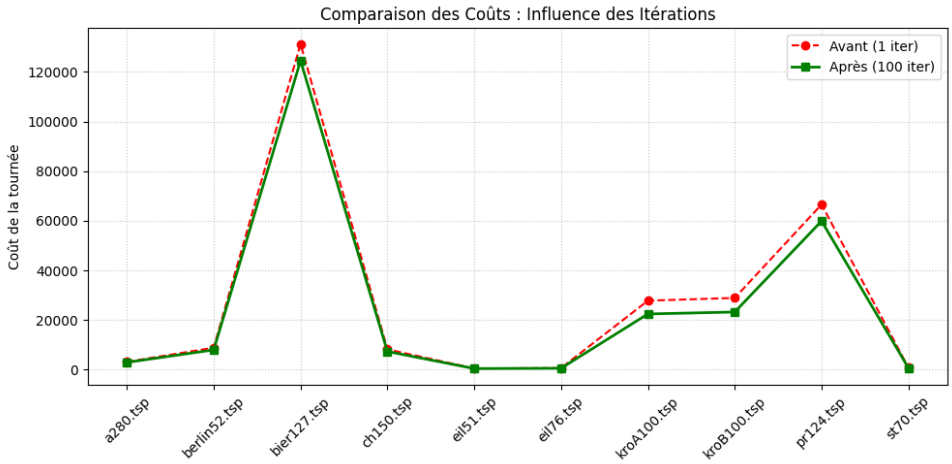
— la température initiale

— le facteur de refroidissement

— le nombre d'itérations par palier

8.1 Influence du nombre d'itérations

Instance	Cout_Avant	Cout_Apres	Amelioration(%)
a280.tsp	3157	2930	7.19
berlin52.tsp	8850	7977	9.86
bier127.tsp	131287	124531	5.15
ch150.tsp	8191	7328	10.54
eil51.tsp	479	438	8.56
eil76.tsp	642	558	13.08
kroA100.tsp	27807	22447	19.28
kroB100.tsp	28912	23236	19.63
pr124.tsp	66570	59897	10.02
st70.tsp	830	703	15.3



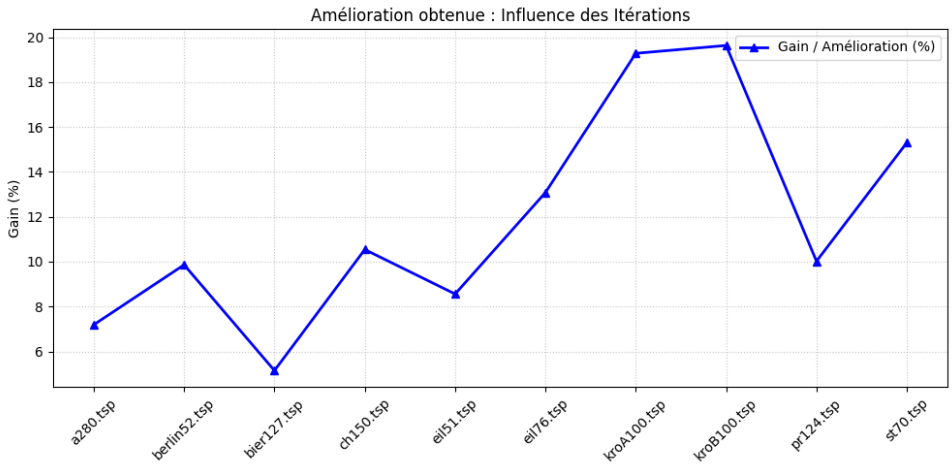
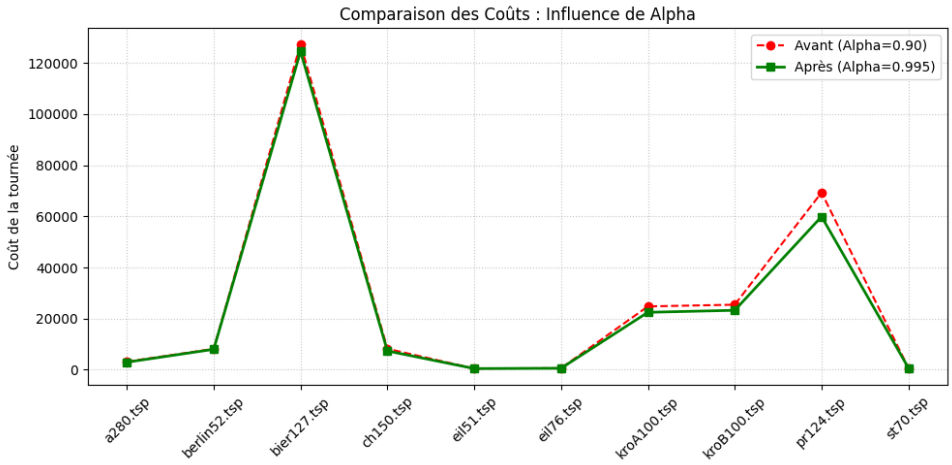


FIGURE 7 – Influence du nombre d'itérations par palier : 1 iter et 1000 iter

8.2 Influence du facteur de refroidissement

Instance	Cout_Avant	Cout_Apres	Amelioration(%)
a280.tsp	3157	2930	7.19
berlin52.tsp	8144	7977	2.05
bier127.tsp	127451	124531	2.29
ch150.tsp	8191	7328	10.54
eil51.tsp	443	438	1.13
eil76.tsp	593	558	5.9
kroA100.tsp	24758	22447	9.33
kroB100.tsp	25432	23236	8.63
pr124.tsp	69297	59897	13.56
st70.tsp	707	703	0.57



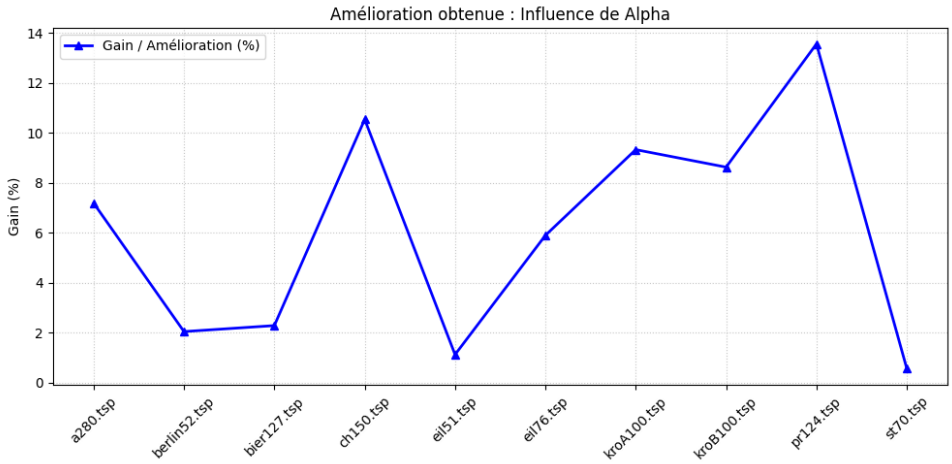


FIGURE 8 – Influence du facteur de refroidissement

Ces analyses permettent d'identifier les paramètres offrant le meilleur compromis entre qualité de solution et temps de calcul.

9. Analyse des résultats

L'analyse des résultats expérimentaux montre que l'heuristique du plus proche voisin permet de générer rapidement une solution initiale de qualité acceptable.

Cependant, cette solution peut être améliorée de manière significative grâce à l'utilisation du recuit simulé.

Le recuit simulé permet d'explorer l'espace des solutions et d'éviter les minima locaux grâce à l'acceptation probabiliste de certaines solutions moins bonnes.

Les résultats montrent également que le choix des paramètres influence fortement les performances de l'algorithme.

10. Conclusion

Dans ce projet, nous avons étudié la résolution du problème du voyageur de commerce en utilisant deux approches complémentaires : une heuristique gloutonne et une métaheuristique basée sur le recuit simulé.

L'heuristique du plus proche voisin permet de générer rapidement une solution initiale, tandis que le recuit simulé permet d'améliorer cette solution en explorant efficacement l'espace des solutions.

Les expérimentations réalisées sur plusieurs instances de la bibliothèque TSPLIB montrent que le recuit simulé permet d'obtenir des solutions de meilleure qualité tout en conservant un temps de calcul raisonnable.

Une étude de l'influence des paramètres du recuit simulé a également permis de mieux comprendre l'impact de ces paramètres sur les performances de l'algorithme.

Ces travaux pourraient être prolongés par l'étude d'autres métaheuristiques ou par l'utilisation de méthodes hybrides combinant plusieurs approches d'optimisation.